

OOP

programare orientată pe obiecte

CURS01: INTRODUCERE

Alexandru Olteanu
{alexandru.olteanu@upb.ro}

Computer Science & Engineering Department
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Universitatea Politehnica București

DE CE SA INVATAM OOP?

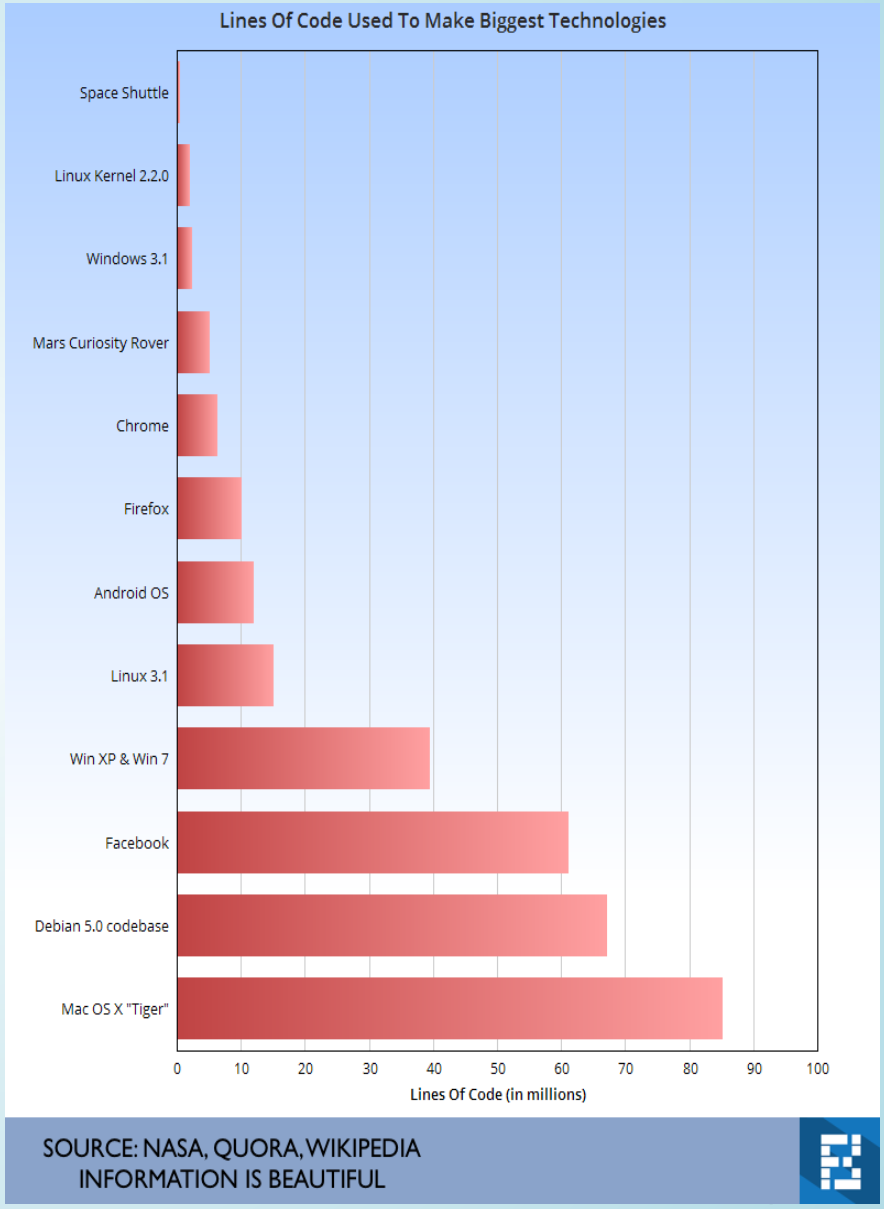
alexandru.olteanu@upb.ro

Cat de mari sunt aplicatiile mari?

(linii de cod, contributori)

Cat de mari sunt aplicatiile mari?
(linii de cod, contributori)

source: fossbytes (2016)



- "The ratio of time spent reading versus writing [code] is well over 10 to 1." *- Robert C. Martin "Uncle Bob" author Clean Code, Agile Manifesto*
- "Code never lies, comments sometimes do" *- Ron Jeffries, author of Extreme Programming books*

- "The ratio of time spent reading versus writing [code] is well over 10 to 1." *- Robert C. Martin "Uncle Bob" author Clean Code, Agile Manifesto*
- "Code never lies, comments sometimes do" *- Ron Jeffries, author of Extreme Programming books*

Ce credeti ca inseamna CLEAN CODE ?

- "Clean code can be read, and enhanced by a developer other than its original author." *- Dave Thomas, godfather of the Eclipse strategy*
- "Clean code does one thing well." *- Bjarne Stoustrup, inventor of C++*
- "Clean code reads like well-written prose." *- Grady Booch, author of Object Oriented Analysis*

CLEAN CODE

- mai usor de inteles
- mai usor de depanat
- mai usor de extins
- mai usor de testat

PROGRAMARE PROCEDURALA

Procedura: O lista de instructiuni care îi spun computerului ce sa fac pas cu pas

PROGRAMARE ORIENTATA-OBIECT

Obiect: componenta a programului care stie cum sa desfasoare anumite actiuni si cum sa interactioneze cu alte elemente din program

```c
struct Point2D {
  int x;
  int y;
}

// ... maybe in different file(s), far far away
void print(Point2D p) {
  printf("(%d,%d)", p.x, p.y);
}
```

```c
class Point2D {
  int x;
  int y;

  void print() {
    cout<<"("<<this.x<<","<<this.y<<")"<<endl;
  }
}

// ... maybe in different file(s), far far away
Point2D p = new Point2D();
p.print();
```

```
printf("Hello");                // C
cout<<"Hello"<<endl;            // C++
System.out.println("Hello");    // Java
```

PROGRAMARE PROCEDURALA

printf este o functie

PROGRAMARE ORIENTATA-OBIECT

- cout este un obiect si operator<< este o metoda a sa
- System.out este un obiect si println este o metoda a sa

PROGRAMARE PROCEDURALA

simplu, reflecta un mod direct de a rezolva problema, ok pentru programe mici

PROGRAMARE ORIENTATA-OBIECT

avansat, modeleaza situatii din realitate, oferind modularitate si ordine în programele mari

PARADIGME DE PROGRAMARE

- imperativ (programatorul descrie pas cu pas rezolvarea problemei)
  - procedural
  - orientat-obiect
  - etc.
- declarativ (programatorul descrie proprietati ale rezultatului dorit, nu cum se calculeaza)
  - functional
  - logic
  - etc.

Nota: limbajele de programare pot avea trasaturi din mai multe paradigme de programare (e.g. in Java 8+ avem clase si expresii lambda)

CUM INVATAM OOP?

alexandru.olteanu@upb.ro

								Laborator	Teme/Test	Deadlines
Octombrie	Lu	Ma	Mi	Jo	Vi	Sb	Du			
Sapt 1	3	4	5	6	7	8	9			Setup environment: wiki page + q&a session la curs
Sapt 2	10	11	12	13	14	15	16	Lab 1 - Java Basics		
Sapt 3	17	18	19	20	21	22	23	Lab 2 - Constructori, referinte		
Sapt 4	24	25	26	27	28	29	30	Lab 3 - Agregare si Mostenire		
Noiembrie	Lu	Ma	Mi	Jo	Vi	Sb	Du			
Sapt 5	31	1	2	3	4	5	6	Lab 4 - Static, final, SIngleton		
Sapt 6	7	8	9	10	11	12	13	Lab 5 - Abstractizare	**Tema**	7 noiembrie - publicare tema - prezentarea sa la prir
Sapt 7	14	15	16	17	18	19	20	Lab 6 - Clase interne		21 noiembrie - deadline (hard)
Sapt 8	21	22	23	24	25	26	27	Lab 7 - Visitor, Polimorfism	**Proiect - etapa 1**	21 noiembrie - publicare etapa 1 proiect - prezentar
Sapt 9	28	29	30	1	2	3	4			
Decembrie	Lu	Ma	Mi	Jo	Vi	Sb	Du			12 decembrie - deadline corectare tema
Sapt 10	5	6	7	8	9	10	11	Lab 8 - Colectii	**Proiect - etapa 1**	12 decembrie - deadline (hard) etapa 1 proiect
Sapt 11	12	13	14	15	16	17	18	Lab 9 - Design Patterns		19 decembrie - publicare etapa 2 proiect - prezentar
Sapt 12	19	20	21	22	23	24	25	Lab 10- Genericitate	**Proiect - etapa 2**	19 decembrie - deadline corectare etapa 1
Vacanta	26	27	28	29	30	31	1			
Ianuarie	Lu	Ma	Mi	Jo	Vi	Sb	Du			
Vacanta	2	3	4	5	6	7	8			
Sapt 13	9	10	11	12	13	14	15	Lab 11 - Java features	**Proiect - etapa 2**	16 ianuarie - deadline hard etapa 2 proiect
Sapt 14	16	17	18	19	20	21	22	Lab 12 - Exceptii	**Test grila - joi/vineri**	23 ianuarie - deadline corectare etapa 2
Sesiune	23	24	25	26	27	28	29		**Sesiune**	

Resurse:

- Moodle CA+CD: forumuri teme
- Moodle CD: slides curs
- Teams CA+CD: anunturi, discutii cu asistentii
- Teams CD: discutii cu asistentii
- OCW: teorie laboratoare, enunturi exercitii laboratoare, enunturi teme
- vmChecker: trimitere solutii teme
- LambdaChecker: trimitere solutii exercitii laborator, problema la examen

INTRO
JAVA SI JVM

alexandru.olteanu@upb.ro

Java Virtual Machine (JVM)

- JVM: masina virtuala ce permite transformarea codului intermediar (Java Bytecode) în instructiuni executabile pe procesorul curent (a.k.a. un program ce execut cod, proiectat pentru portabilitate, trade-off de performanta)
- Mai multe limbaje de programare folosite pentru a produce cod ce ruleaza in JVM: Java, Scala, Kotlin, Clojure, Groovy etc. (20+ limbaje)

Java Platforms:

- Java SE: basics, networking, security, database access, GUI
- Java EE: developing and running large-scale network applications
- Java ME: small-footprint virtual machine for small devices
- Java FX: use hardware-accelerated graphics and media engines

Java environments:

- JRE: Java Runtime Environment
- JDK: Java Development Kit (include JRE)

Automatic translation tools between programming languages (Java, C#, Kotlin)

alexandru.olteanu@upb.ro

Reading & Watching material:

- Simon Sinek - Start with Why
- Clean Code, Robert C. Martin, 2008
- Effective Java, 3rd Edition, Joshua Bloch, 2018
- LambdaChecker
- ocw.cs.pub.ro/courses/poo-ca-cd > Regulament
- ocw.cs.pub.ro/courses/poo-ca-cd > Lab01